

Sara White

CSD-380

Assignment 3.2 – Version Control Guidelines

Version control systems (VCSs) are necessary for managing code development and changes. While version control systems are especially helpful for tracking changes and coordinating workflows for software development teams, they are also useful for independent programmers working alone or in very small teams. Working within an environment that clearly documents the source code's evolution throughout the software development lifecycle supports code integrity and collaboration while providing the structure and organization necessary for efficient workflows. Version control guidelines are effective when they help teams maintain a stable codebase, reduce merge conflicts and siloes and encourage secure coding practices. I reviewed guidance from GitLab, Atlassian and GitHub to compare common best practices recommended by industry experts.

GitLab's version control recommendations are focused on day-to-day quality of commits and branches. GitLab suggests making small, incremental changes instead of committing several different unrelated edits at once. They also emphasize the importance of atomic commits, meaning that each commit should represent a single logical change. GitLab recommends writing descriptive commit messages so other developers can easily understand the purpose of the change. They provide an important guideline when it comes to maintaining code integrity: development should

take place on branches instead of directly on the main source code and it should be reviewed before merging.

Atlassian's source code management guidance makes many of the same points as GitLab. They also recommend committing small, incremental changes with clear commenting as well as branch-based development to support a healthy codebase. Atlassian emphasizes the importance of working from the latest versions of the repository before making changes to avoid merge conflicts and asynchronous workflows.

GitHub's guidance covers some of the same ground as GitLab and Atlassian, but it focuses more on specific documentation practices. For example, they recommend creating a README file and related repository documentation so that contributors have a clear understanding of the project's purpose and setup requirements. One process that GitHub addressed that I didn't see discussed in the GitLab and Atlassian guidelines was the preference for branching over forking. Branching creates a parallel line of development inside of the same repository, while forking makes a whole, separate copy of the repository under a different user account. Forking is a useful process for outside contributors because it provides a way for them to interact with the code safely, but for regular members of the development team it can create unnecessary moving parts and fragment the workflow.

My list would include many of the same points from Atlassian, GitHub and GitLab, but I'd add some extra points to address AI-assisted contributions to a codebase. AI is a powerful tool, but AI-assisted code should be treated as an untrusted external contribution until it has been reviewed by a human developer. In practice, this

means AI-generated changes should be isolated in a feature branch, or other controlled workflow, rather than being committed directly to a protected branch. If an autonomous AI agent is contributing code, their work should be forked and then selectively merged. This approach helps ensure that AI-generated code is tested and reviewed for security and correctness before it becomes part of the shared codebase.

Here's what my list would look like:

1. Make small, incremental changes instead of bundling lots of unrelated edits into one big commit. Small changes are easier to review, test, and roll back if something breaks.
2. Make small changes and commit those changes in groupings that make sense to avoid unrelated edits being committed at the same time.
3. Make frequent commits so changes are documented in real time
4. Write clear, descriptive messages with commits. It's better to over-explain than to leave incomplete or vague messages. This not only helps other developers on your team, but it will help future versions of you if you have to revisit them.
5. Use a clear branching strategy and ensure all developers follow the process as closely as possible to create consistent and predictable workflows.
6. Work from the latest version of the repository before starting new changes.
Pulling or fetching recent updates helps prevent avoidable conflicts and keeps your local work synchronized with the rest of the team.
7. Use code review or pull request review before merging important changes.

8. Create a clear README and basic repository documentation so the intent of the project is clearly defined. This helps contributors added down the line, but it also helps the current development team understand what they are working toward.
9. Secure the repository - use repository security features where possible.
10. Track dependencies and watch for vulnerabilities in third-party packages. Be aware of the software supply chain to understand how far-reaching your responsibilities actually are.
11. Use Git LFS or another other file management tools for large binary files.

References

Atlassian. (n.d.). Source code management.

<https://www.atlassian.com/git/tutorials/source-code-management>

Dennis, Nzioki (2024, March 25). Mastering Version Control: A Developer's Guide.

<https://dev.to/nziokidennis/mastering-version-control-a-developers-guide-e1e>

GitHub. (n.d.). Best practices for repositories. <https://docs.github.com/en/enterprise-cloud@latest/repositories/creating-and-managing-repositories/best-practices-for-repositories>

GitHub. (n.d.). Best practices for securing code in your supply chain.

<https://docs.github.com/en/code-security/tutorials/implement-supply-chain-best-practices/securing-code>

Gitlab. (n.d.). What are Git version control best practices?

<https://about.gitlab.com/topics/version-control/version-control-best-practices/>

Mosyan, D. (2023, December 28). Version Control Branching Strategies. Medium.

<https://medium.com/@dmosyan/version-control-branching-strategies-e68e8d5ef1e0>